

Find Paths Between Gateway Persons and a Prospect Company

This tutorial shows how to use the **Gateways** REST API to retrieve paths between potential gateway persons and people associated with a prospect company.

The tutorial uses the documented `GET /gateways` endpoint and demonstrates how to supply the required parameters, control the number of returned paths, enable debug information, and interpret the response.

Before you start

You need:

- A prospect company ID
- One or more potential gateway person IDs
- Access to the Gateways REST API

The API is documented as a REST API using the `GET` method.

Step 1: Identify the prospect company

The `prospect_id` parameter identifies the prospect company.

It is a required string parameter.

```
prospect_id={prospectId}
```

For example:

```
prospect_id=123456
```

Use the prospect company ID required by your application environment.

Step 2: Specify potential gateway persons

Use `potential_gateways` to provide the IDs of the people who may act as gateways to the prospect company.

The parameter is required and accepts a comma-separated list of strings.

```
potential_gateways={gatewayId1},{gatewayId2}
```

For example:

```
potential_gateways=1667,56787
```

The original API example demonstrates multiple gateway IDs supplied in a single request.

Step 3: Control the number of paths

The API provides optional parameters for controlling the number of results.

Paths per gateway person

Use `number_of_path_per_gw_person` to specify the maximum number of paths returned for each gateway person.

The default value is `10`.

```
number_of_path_per_gw_person=10
```

Gateway persons per prospect person

Use `number_of_gw_person_per_prospect_person` to specify the maximum number of gateway persons returned for each prospect person.

The default value is `10`.

```
number_of_gw_person_per_prospect_person=10
```

Number of prospect persons

Use `number_of_prospect_person` to specify the maximum number of prospect persons returned.

This parameter is optional and does not have a documented default value in the source material.

Step 4: Enable debug information

The optional `debug` parameter enables debug information.

Its default value is `false`.

To enable debug mode:

```
debug=true
```

When debug mode is enabled, the response can include formatted path information and the path-computation time.

Step 5: Build the request

Combine the parameters into a `GET` request.

```
GET /gateways?prospect_id={prospectId}&potential_gateways={gatewayId1},  
{gatewayId2}&debug=true
```

The documented API path also supports the optional result-limit parameters:

```
GET /gateways?prospect_id={prospectId}&potential_gateways={gatewayId1},  
{gatewayId2}  
&number_of_path_per_gw_person=10&number_of_gw_person_per_prospect_person=10&number_of_prospect  
&debug=true
```

Step 6: Send the request

Send the `GET` request to the Gateways endpoint using the appropriate API environment.

The source material does not document authentication, request headers, base URLs or a specific API client, so those details are outside the scope of this tutorial.

Step 7: Check the response

A successful request returns:

```
Status code: 200  
Content type: application/json
```

The response contains information about prospect persons and the gateway persons and paths associated with them.

A simplified response structure is:

```
{  
  "list_of_prospects": {  
    "link": "PERSON/{personId}",  
    "score": 1,  
    "gateways_persons": [  
      {  
        "link": "PERSON/{gatewayPersonId}",  
        "score": 1,  
        "paths": [  

```

```

    {
      "link": "Company/{companyId}"
    },
    {
      "link": "JOB/{jobId}"
    },
    {
      "link": "PERSON/{personId}"
    }
  ]
}
]
}
}
}

```

Step 8: Interpret the paths

Each path represents a sequence of linked entities.

The documented example shows a path containing links to:

1. A company
2. A job
3. A person

For example:

```

Company/{companyId}
  ↓
JOB/{jobId}
  ↓
PERSON/{personId}

```

The response can also include a score associated with the prospect person and gateway person.

Step 9: Use debug information when needed

When `debug=true`, the response can provide additional information such as:

- `paths_string` — formatted path information
- `path_time_in_ms` — path-computation time in milliseconds

This can help when examining how paths were calculated or reviewing the returned path information.

Troubleshooting

400 Bad Request

A `400` response can occur when:

- no company exists for the supplied `prospect_id`, or
- one or more parameters contain an invalid value.

Check the prospect ID and the supplied parameter values before sending the request again.

Next steps

For the complete parameter definitions and response structure, see the [Gateways API Reference](#).